

CSC490 Assignment A1: Project Landscape

Ananya Jain (1007706573), Aviral Bhardwaj (1008239407)
Ansh Agrawal (1007413103), Mann Thakkar (1007647744)

1 Interest Statements

Team Problem Statement

Our team is drawn to the intersection of Dynamic Neural Architectures and Bio-Inspired Computing. We are motivated by a specific frustration with current AI scaling: to make models smarter, we usually just make them bigger, but the resulting architectures are static. We are specifically looking at Mixture-of-Experts (MoE) models. While MoEs are efficient, they suffer from “expert collapse” where the model favors just a few experts forcing engineers to use artificial “auxiliary loss” functions to balance the load. We want to build the Thermodynamic Mixture-of-Experts (T-MoE), a system that doesn’t need a central router to tell it what to do. Instead, we want to prove that simple biological rules like experts getting “tired” (fatigue) or splitting into two when overwhelmed (fission) can naturally solve the problems of catastrophic forgetting and load balancing. We attempt to move away from rigid, pre-planned architectures to a model that “grows” its own topology based on the difficulty of the task.

Individual Interest Statements

- **Ananya Jain:** I will focus on integrating **Reinforcement Learning (RL)** into the system. My objective is to explore whether RL can enable experts to **self-organize and cooperate organically**, rather than relying solely on supervised learning. In particular, I am interested in leveraging **GRPO-style techniques** to investigate whether hyperparameter adaptation can be automated, allowing the model to dynamically adjust to different task domains.
- **Aviral Bhardwaj:** I will lead the design and implementation of the **deep learning architecture**. My goal is to develop custom model layers that enable the network to **dynamically instantiate and manage new experts**. This includes injecting our proposed modules into open-source LLMs, debugging tensor shapes and interfaces, and iteratively refining the architecture to ensure functional and computational correctness.
- **Ansh Agrawal:** I will concentrate on the **mathematical formulation** of the proposed architecture. This involves translating abstract concepts such as *metabolic energy* and *fatigue* into **stable, differentiable equations**. Additionally, I will contribute to defining appropriate **evaluation metrics** (e.g., perplexity, entropy-based measures) to quantitatively assess learning and coordination behavior.
- **Mann Thakkar:** I will take primary responsibility for **infrastructure and data pipelines**. My focus is on the engineering challenges of **training and scaling large models across multiple GPUs**, ensuring that the training loop is efficient, stable, and tractable. I will also work on performance optimization, including the use of **custom CUDA kernels** where appropriate.

2 Landscape Analysis

Relevant Item	Description	Commentary
Paper: DynMoE	Dynamic gating mechanism that automatically tunes the number of active experts per token (Top-Any) based on input complexity rather than a fixed Top-K.	Closest to our concept, but relies on reactive thresholds instead of metabolic fatigue and cannot physically spawn new experts.
Paper: MoE-MLA-RoPE	Unified architecture integrating Multi-head Latent Attention (MLA) for KV-cache compression and Rotary Positional Embeddings (RoPE) into a massive sparse MoE.	Validates MoE efficiency, but treats memory as static; lacks the multi-tier “Resurrection” logic needed to retrieve old experts for lifelong learning.
Paper: MoE++	Heterogeneous MoE with zero-computation experts (skip/copy) for efficiency.	Accelerates inference by skipping layers, but does not provide a mechanism for structural growth.
Paper: MoR	“Mixture of Routers” utilizing sub-routers to optimize joint expert selection.	Improves PEFT performance on specific tasks but increases routing latency and compute requirements.
Paper: MoT	“Mixture of Tokens” which mixes token representations across the batch dimension.	Reduces training instability in sparse models but introduces significant cross-node communication overhead.
Paper: MoE-Lens	Comprehensive analysis of MoE scaling laws and hardware-level bottleneck modeling.	Identifies CPU/GPU memory trade-offs, providing the hardware rationale for our Genetic Archive.
Paper: Mixtral 8x7B	High-performance sparse MoE that popularized the Top-2 gating strategy.	Current industry baseline; suffers from catastrophic forgetting when shifted to significantly new domains.
Paper: S-LoRA	A high-throughput serving engine designed to manage thousands of LoRA adapters concurrently.	Solves the “Expert Multiplicity” memory problem but is limited to inference; does not handle training.
DeepSeek-V3	SOTA Open-MoE (671B) using auxiliary-loss-free load balancing and Multi-head Latent Attention (MLA).	Proves MoE efficiency at scale, but experts are static; lacks dynamic mitosis/fusion capabilities.
Hugging Face TRL	The standard library for Reinforcement Learning (PPO, DPO, GRPO) on transformer models.	Excellent ecosystem support, but lacks native integration for MoE-specific metabolic reward functions.

3 Project Outline

3.1 Problem Statement

Traditional MoE architecture rely on learned gating networks that frequently suffer from Representation Collapse over-specializing small subset of experts since they receives most of tokens, leading to under-utilized experts, and wasted capacity. To combat this, industry standards currently rely on artificial auxiliary balancing losses. However, when the tasks distribution shifts it leads to catastrophic forgetting.

3.2 Proposed Solution (High-level Technical Approach)

We propose T-MoE, a framework that replaces current router with **Similarity-Fatigue** mechanism (aka *Metabolic Routing Engine*): alignment to an expert i minus its current fatigue level F_i

- **Natural Load Balancing:** Fatigue increases with usage and decays over time, preventing any single expert from monopolizing tokens and forcing the model to explore “rested” experts.
- **Elastic Expert Selection:** We replace Top-K with dynamic-K MoE with a Learnable Threshold.

Each expert is a LoRA adapter on which we perform:

- **Mitotic Fission:** Over longer intervals, the model splits overloaded experts (high fatigue) into two specialized child LoRA adapters. We start with 8 experts and cap at 64 experts.
- **Entropic Apoptosis:** Prunes rarely used ones if they fall below a metabolic usage threshold via block-wise GPU-friendly updates and Free-Energy objective is optimized using Group Relative Policy Optimization (GRPO)

Key technology:

- Core Logic: Python, PyTorch, Weights & Biases (model training tracking)
- LLM Models: GPT-2 (local dev), Llama 3x8B, Qwen 3-8B, Mixtral 8x7B,
- Infrastructure: RTX A4500 (local dev); AWS P5.48xlarge for open source models testing.

3.2.1 Milestones:

Phases	Timeline	Checkpoints
Foundation	Jan 21 – Feb 1	Zero-Aux Metabolic Routing: Initial Setup/ Adding routing engine.
Adaptation	Feb 2 – Feb 15	Informational Stress Triggers: Dynamic growth (Mitosis) and death (Apoptosis) based on loss-variance ($\mathcal{S}_i$).
Consolidation	Feb 16 – Feb 23	Entropic Fusion: Consolidates redundant expert weights (θ_{new}).
Autonomy	Feb 24 – Mar 10	RL Homeostasis: GRPO-learned “Existence Tax” (β) and long-term disk serialization for memory storage.
Scaling	Mar 11 – Mar 29	Testing on Open-Source: Scaling to 8B+ Llama-3/Qwen-3 models.

3.2.2 Unknowns to Investigate

- Does Metabolic Fatigue provide better Expert Specialization than SOTA at scale?
- Does the Cosine Similarity threshold scale linearly from GPT-2 to Qwen/Llama?
- Does adding a new dynamic router affects the latency of the outputs if ran in production?

4 Project Press Release

TORONTO, ON – JANUARY 23, 2026: T-MoE reduces the cost of maintaining high-performance specialized AI models through an autonomous, self-thinning architecture that expands and shrinks based on task complexity.

Enterprises building specialized AI applications are currently forced to choose between two failures: “generalist” models that lack domain-specific accuracy, or “static MoE” models that are prohibitively expensive to run. As businesses add more specialized knowledge, their infrastructure costs scale linearly, and the models become “stiff” failing to adapt to new data without manual, costly retraining. Today, developers spend more time managing GPU memory and load-balancing experts than improving their models’ actual intelligence.

T-MoE (Thermodynamic Mixture-of-Experts) is the first AI architecture that manages its own hardware resources. By replacing fixed selection rules with an “Elastic Brain,” T-MoE automatically activates more processing power for complex reasoning and shuts down redundant components for simple tasks. It allows companies to scale to hundreds of specialized “experts” without increasing their serving costs, achieving sub-millisecond latency and significant reductions in VRAM usage.

Existing solutions use “Top-K” routing, which forces a model to use a fixed amount of power regardless of whether it is answering a simple math question or a complex legal query. T-MoE introduces “Metabolic Optimization.” It treats compute as a finite resource, using a learnable threshold to prune inactive pathways in real time. Unlike static models that require manual pruning, T-MoE’s “Mitotic Fission” allows it to grow new experts as it learns, and “Entropic Apoptosis” allows it to discard what it no longer needs all without human intervention.

“The challenge with traditional Mixture-of-Experts has always been the ‘Density Tax’ the reality that model capacity scales faster than hardware efficiency. With T-MoE the routing stays balanced without relying on an extra load-balancing loss, and metabolic system that intelligently resizes itself based on the complexity of the prompt is truly novel.”

— John Doe, Research Scientist (Google Deepmind)

Getting started is easy: developers drop the T-MoE router into their existing LoRA-based training pipeline. The system begins by observing token “difficulty” and automatically sets a metabolic threshold. Within the first hour of training, the model begins to “self-thin,” identifying redundant experts and consolidating their knowledge. By the time the model is deployed, the customer sees an immediate reduction in active parameter count while maintaining the original model’s accuracy. The result is a specialized model that runs at the speed and cost of a much smaller generalist.

T-MoE is designed for ML infrastructure teams and product engineers at mid-to-large enterprises who need to deploy multiple specialized AI features (e.g., multi-language support, multi-department coding assistants) on a fixed hardware budget.

Visit our documentation at tmoe.ai/alpha to request early access and read more about here at [arXiv:XXX](#) (Published at XXX conference).

A Appendix: Iterations

A.1 Idea Iteration

In lecture 2 of CSC490, we were shown a graph that although many modern LLMs advertise context windows of 1M–2M tokens, their effective usable context is significantly smaller. Beyond a certain length, models tend to hallucinate, lose coherence, and suffer from catastrophic forgetting. Additionally, while using CodingIDEs, we noticed that once the conversation context is largely exhausted, systems often rely on internal mechanisms to compress or summarize prior context in order to continue the interaction.

Motivated by these observations, our initial idea was to design a system in which multiple experts share a common context and cooperate to manage it. The intuition was that if a particular expert is queried frequently or proves especially relevant, its contextual information should be prioritized, while less useful context is deprioritized or discarded.

Given our interest in Reinforcement Learning (RL), we then evolved this idea from cooperative sharing to a competitive *bidding-based* mechanism. In this formulation, each expert bids against others to retain its information within the limited context window, with an RL loop governing and optimizing the bidding dynamics. While exploring prior work in this direction, we encountered the *Mixture of Bidders* framework, which addressed a similar problem through competitive allocation.

Building on this insight, we further refined our approach. Instead of explicitly bidding for context, we shifted toward a mechanism that actively retains useful experts while pruning ineffective ones. At this stage, we also broadened the scope of the problem: rather than focusing narrowly on context window management, we aimed to design a more general framework applicable across tasks and input distributions.

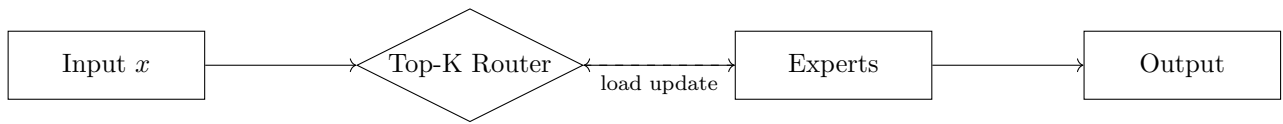
This led us to study existing routing mechanisms in Mixture-of-Experts (MoE) architectures. Ultimately, we arrived at our final idea: to redesign the LLM routing mechanism itself so that it is no longer static, but instead **dynamic and adaptive**, capable of adjusting expert selection and allocation based on incoming tasks and token-level signals.

A.2 Important Acronyms

MoE (Mixture of Experts)	A machine learning architecture that uses multiple specialized sub-networks (“experts”) to process different parts of the input data. This allows models to scale parameters significantly (e.g., GPT-4, Mixtral) while keeping inference costs lower by only activating a subset of experts per token.
LoRA (Low-Rank Adaptation)	A parameter-efficient fine-tuning (PEFT) technique that freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture. This drastically reduces the number of trainable parameters for downstream tasks.
GRPO (Generalized Reinforcement Policy Optimization)	A reinforcement learning–based optimization framework that learns control policies over continuous and discrete system parameters. In the context of T-MoE, GRPO governs metabolic variables (e.g., routing thresholds, expert activation costs, and structural adaptation rates), enabling stable self-tuning of expert selection and compute allocation without explicit load-balancing losses.

A.3 High Level Diagram

A.3.1 Standard MoE:



A.3.2 T-MoE

